

Curs 8 — Structuri de Date (Gabriel Istrate)

Sinteză: Perechea cea mai apropiată + Augmentarea structurilor de date

🎯 = punct de examen, ⚠️ = capcană, ★ = fapt fundamental.

Cursul are două părți distincte: (1) **closest pair** — încheierea geometriei, un divide-and-conquer elegant; (2) **augmentarea** arborilor red-black ca să suporte operații noi — order statistics și interval trees. Ideea-fir a părții a doua: nu inventezi o structură nouă, ci **adaugi un câmp** la una existentă și ai grijă să-l menții.

PARTEA 1 — Perechea de puncte cea mai apropiată (closest pair)

1.1 Problema și complexitățile

Dat un set de n puncte, găsește perechea cu cea mai mică distanță euclidiană.

- **Forță brută:** $\Theta(n^2)$ — compari toate perechile.
- **Divide-and-conquer:** $O(n \log n)$.

1.2 Algoritmul divide-and-conquer

Pregătire (o singură dată): **presortezi** punctele în două tablouri:

- (X) = punctele sortate crescător după x ;
- (Y) = punctele sortate crescător după y .

⚠️ Cheia eficienței: **nu îți permiti să sortezi din nou în fiecare apel recursiv** (ar strica limita). Sortezi o dată la început.

Apoi, recursiv pe subsetul (P) :

- **Cazul de bază:** dacă $(|P| \leq 3)$, forță brută (câteva comparații, $O(1)$).
- **Divide:** găsești o **dreaptă verticală** (l) care împarte (P) în două jumătăți egale: (PL) (toate punctele la stânga) și (PR) (toate la dreapta), cu $(|PL| = \lceil |P|/2 \rceil)$, $(|PR| = \lfloor |P|/2 \rfloor)$. Împarți și tablourile: (XL/XR) , (YL/YR) .
- **Conquer:** apeluri recursive pe (PL, XL, YL) și (PR, XR, YR) , care întorc cele mai mici distanțe din fiecare jumătate, (δ_L) și (δ_R) .
- **Combine:** $(\delta = \min(\delta_L, \delta_R))$. Acum trebuie verificat dacă există o pereche „transfrontalieră” (un punct în (PL) și unul în (PR)) la distanță $(< \delta)$.

1.3 Trucul din pasul de combinare (partea genială)

Dacă există o pereche transfrontalieră mai apropiată decât δ , **ambele puncte trebuie să fie în banda de lățime 2δ din jurul dreptei l** (câte δ de fiecare parte). Motivul: dacă un punct ar fi mai departe de δ de l , ar fi deja la distanță $> \delta$ față de orice punct de cealaltă parte, doar pe axa x.

Deci:

1. Construiești Y' = tabloul Y din care scoți toate punctele din afara benzii 2δ , păstrat sortat după y.
2. Pentru fiecare punct p din Y' , cauți puncte la distanță $< \delta$ — dar **★ e suficient să verifici următoarele 7 puncte** care îl urmează pe p în Y' .
3. Calculezi cea mai mică astfel de distanță δ' . Dacă $\delta' < \delta$, ai găsit o pereche mai bună; altfel δ rămâne răspunsul.

★ De ce doar 7 puncte (faptul testabil)

Pentru fiecare punct, consideri **dreptunghiul $\delta \times 2\delta$** centrat pe dreapta l . În acest dreptunghi încap **cel mult 8 puncte** — fiindcă în fiecare jumătate (de lățime δ) punctele sunt la distanță $\geq \delta$ unul de altul (altfel recursivitatea le-ar fi găsit deja), deci nu pot fi prea multe. Punctul p fiind unul dintre ele, mai rămân cel mult 7 de verificat după el. Asta face pasul de combinare $O(n)$ în loc de $O(n^2)$.

1.4 Complexitatea și problema sortării

Recurența: $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$ (master theorem, cazul echilibrat ca la mergesort).

⚠ **Dificultatea principală:** să menții (XL, XR, YL, YR, Y') sortate fără să re-sortezi.

Observația cheie: în fiecare apel vrei să formezi un **subset sortat dintr-un tablou sortat**, împărțindu-l în două jumătăți. Asta e **inversul operației MERGE din MERGESORT** — în loc să combini două tablouri sortate într-unul, **împarți** un tablou sortat în două sortate, printr-o singură trecere $O(n)$.

```
// Împărțirea lui Y în YL și YR (păstrând sortarea după y)
length[YL] = length[YR] = 0
for i = 1 to length[Y]
    if Y[i] < PL
        length[YL]++ ; YL[length[YL]] = Y[i]
    else
        length[YR]++ ; YR[length[YR]] = Y[i]
```

Cum obții tablourile sortate inițial? **Presort**, $\Theta(n \log n)$ — o singură dată, la început.

1.5 🌀 Fapte de examen — closest pair

- Forță brută $\Theta(n^2)$; divide-and-conquer $O(n \log n)$.
 - Banda de lățime **2δ** în jurul drepte de separare.
 - **Cel mult 8 puncte** în dreptunghiul $\delta \times 2\delta \rightarrow$ verifici doar **7 puncte** următoare.
 - Recurența $T(n) = 2T(n/2) + O(n) = O(n \log n)$.
 - Sortarea se face prin **presort** + împărțire (inversul lui MERGE), nu re-sortare în fiecare apel.
-

PARTEA 2 — Augmentarea structurilor de date

2.1 Ideea generală

Ce faci când nicio structură existentă nu-ți acoperă nevoile? Invenți una nouă, sau — mai realist în practică — **modifici ușor o structură „standard”** ca să suporte operații noi, **stocând informație suplimentară** în ea. ⚠️ Nu e mereu simplu: **informația nouă trebuie actualizată și menținută** de operațiile structurii (insert, delete, rotații).

Două exemple, ambele prin modificarea arborilor red-black:

1. **Order statistics** — al i-lea element / rangul unui element;
2. **Interval trees** — o mulțime de intervale.

Plus o **teoremă generală** despre când augmentarea e „gratuită”.

2.2 🌀 Cei 4 pași ai augmentării (de memorat)

1. **Alegi structura de bază.**
2. **Determini informația suplimentară de menținut.**
3. **Verifici** că informația poate fi menținută în operațiile structurii.
4. **Dezvolți operațiile noi** cerute de câmpurile noi.

2.3 Order Statistics Tree (arbore de statistici de ordine)

Un **red-black tree** cu un câmp în plus per nod: $\boxed{\text{size}}$ = numărul de noduri din subarborele cu rădăcina în acel nod.

- Câmpuri: $\boxed{\text{key, color, p, left, right, size}}$.
- $\boxed{\text{size}[\text{nil}] = 0}$.

- ★ $\text{size}[x] = \text{size}[\text{left}[x]] + \text{size}[\text{right}[x]] + 1$.

Suportă două operații noi, ambele $O(\log n)$:

OS-SELECT(x, i) — al i-lea cel mai mic element

```
OS-SELECT(x, i)
1  r = size[left[x]] + 1      // rangul lui x în subarborele său
2  if i == r
3      return x
4  elseif i < r
5      return OS-SELECT(left[x], i)
6  else
7      return OS-SELECT(right[x], i - r)
```

Logica: $r = \text{size}[\text{left}[x]] + 1$ e rangul lui x în propriul subarbor (câte elemente $\leq x$ sunt = mărimea subarborelui stâng + el însuși, din proprietatea BST). Dacă $i = r$, x e răspunsul. Dacă $i < r$, al i-lea e în stânga (cu același i). Dacă $i > r$, e în dreapta, dar cauți al $(i - r)$ -lea acolo (fiindcă r elemente — stânga + x — vin înaintea subarborelui drept). Timp $O(\text{înălțime}) = O(\log n)$.

OS-RANK(T, x) — rangul lui x în tot arborele

```
OS-RANK(T, x)
1  r = size[left[x]] + 1
2  y = x
3  while y ≠ root[T]
4      if y == right[p[y]]
5          r = r + size[left[p[y]]] + 1
6      y = p[y]
7  return r
```

Logica: pornești cu rangul lui x în propriul subarbor. Urci spre rădăcină: de câte ori y e copil **drept**, tot ce e în subarborele stâng al părintelui (plus părintele) vine înaintea lui x , deci adaugi $\text{size}[\text{left}[p[y]]] + 1$. **Invariant:** la începutul fiecărei iterații, r e rangul lui x în subarborele cu rădăcina y . Ajungi la rădăcină → rangul global. Fiecare pas $O(1)$, y urcă → $O(\log n)$.

Menținerea câmpului size

- Inserare**, faza 1 (coborâre + inserare frunză): fiecare nod pe drumul de la rădăcină la noul nod își crește size cu 1; noul nod primește $\text{size} = 1$. $O(\log n)$.

- **Inserare**, faza 2 (recolorări + rotații): rotațiile schimbă structura, deci trebuie actualizat `size`. La `LEFT-ROTATE(x)` adaugi:

```
size[y] = size[x]
size[x] = size[left[x]] + size[right[x]] + 1
```

`RIGHT-ROTATE` simetric.

- **Ștergere**: analog — actualizezi `size` pe drumul de la nod în sus (decrementezi cu 1), plus rotațiile ca la inserare.

★ De ce `size` și nu direct rangul (insight de design)

Dacă ai stoca în fiecare nod **rangul** lui direct, `OS-SELECT`/`OS-RANK` ar fi rapide — DAR inserarea unui element mai mic decât toate ar schimba rangul **tuturor** nodurilor → actualizare $O(n)$. Stocând în schimb **mărimea subarborelui**, o modificare se propagă **doar la strămoși** ($O(\log n)$ noduri), deci inserarea/ștergerea rămân $O(\log n)$. Åsta e exact motivul pentru care alegi `size`.

2.4 🌀 Teorema de augmentare (rezultatul general)

Teoremă: Fie f un câmp care augmentează un RB tree cu n noduri. Dacă $f[x]$ poate fi calculat în $O(1)$ folosind doar informația din x , `left[x]` și `right[x]` (inclusiv `f[left[x]]` și `f[right[x]]`), atunci putem menține f în toate nodurile în timpul inserării și ștergerii **fără a afecta performanța $O(\log n)$** .

Ideea demonstrației: o modificare a câmpului f într-un nod x se propagă **doar la strămoșii lui x** în arbore (doar subarborii strămoșilor conțin nodul modificat). Inserarea/ștergerea ating $O(\log n)$ noduri și fac $O(\log n)$ rotații, fiecare reparabilă în $O(1) \rightarrow O(\log n)$ total.

★ Condiția cheie e „ f **calculabil din informație locală**” (nodul + cei doi copii). `size` și `max` o satisfac; „rangul stocat direct” NU (rangul depinde de poziția globală, nu de copii).

2.5 Interval Trees (arbori de intervale)

Definiții:

- Interval închis `[t1, t2]` (sau deschis/semi-deschis). `low[i] = t1`, `high[i] = t2`.
- ★ i și i' **se suprapun** dacă `low[i] ≤ high[i']` ȘI `low[i'] ≤ high[i]`.
- **Trichotomia intervalelor:** pentru orice două intervale, exact una din trei: (1) se suprapun; (2) i e la stânga lui i' (`high[i] < low[i']`); (3) i e la dreapta lui i' (`low[i] > high[i']`).

Operații: `INTERVAL-INSERT`, `INTERVAL-DELETE`, și `INTERVAL-SEARCH(T, i)` → întoarce un nod al cărui interval se suprapune cu `i`, sau `nil`.

Implementarea (RB tree augmentat)

1. RB tree, fiecare nod `x` stochează un interval `int[x]`; **cheia e capătul stâng:** `key[x] = low[int[x]]`.
2. Câmp suplimentar: ★ `max[x]` = cea mai mare valoare a oricărui capăt de interval din subarboarele cu rădăcina `x`.
3. Întreținere: `max[x] = max(high[int[x]], max[left[x]], max[right[x]])`. (E calculabil din nod + copii → teorema garantează $O(\log n)$.)

INTERVAL-SEARCH

```
INTERVAL-SEARCH(T, i)
1  x = root[T]
2  while x ≠ nil[T] and i nu se suprapune cu int[x]
3      if left[x] ≠ nil[T] and max[left[x]] ≥ low[i]
4          x = left[x]
5      else
6          x = right[x]
7  return x
```

★ **Regula de direcție:** dacă subarboarele stâng are `max[left[x]] ≥ low[i]`, **mergi stânga**; altfel **mergi dreapta**. Un singur drum de la rădăcină → $O(\log n)$.

De ce e suficient un singur drum (corectitudinea — „direcția sigură”)

Invariant: dacă arborele `T` conține un interval care se suprapune cu `i`, atunci există unul și în subarboarele cu rădăcina `x`. Deci urmărind un singur drum nu pierzi soluția:

- **Mergi dreapta** (`max[left[x]] < low[i]`): toate intervalele din stânga se termină înainte ca `i` să înceapă (capătul lor maxim $< low[i]$), deci **niciunul** nu se poate suprapune cu `i` → dacă există o suprapunere, e în dreapta. Sigur.
- **Mergi stânga** (`max[left[x]] ≥ low[i]`): există un interval `i'` în stânga cu `high[i'] = max[left[x]] ≥ low[i]`. Dacă stânga totuși nu conține nicio suprapunere, atunci (din trichotomie: cum `i` și `i'` nu se suprapun dar `high[i'] ≥ low[i]`, rezultă `high[i] < low[i']`) niciun interval din dreapta — care au `low ≥ low[i'] > high[i]` — nu se poate suprapune cu `i` either. Deci a merge stânga e sigur.

2.6 🌟 Fapte de examen — augmentare

- **Order statistics tree** = RB tree + câmp `size`; `size[x] = size[left] + size[right] + 1`.

- **OS-SELECT** (al i-lea element) și **OS-RANK** (rangul), ambele $O(\log n)$.
- **Teorema:** dacă $f[x]$ se calculează din x +copii în $O(1)$, se menține în $O(\log n)$; modificarea se propagă doar la **strămoși**.
- **Interval tree** = RB tree + câmp **max** (capătul maxim din subarbore); cheia = **low** al intervalului.
- **INTERVAL-SEARCH**: mergi **stânga** dacă $\max[\text{left}[x]] \geq \text{low}[i]$, altfel dreapta; $O(\log n)$, un singur drum.
- Suprapunere: $\text{low}[i] \leq \text{high}[i']$ și $\text{low}[i'] \leq \text{high}[i]$. Trichotomie: suprapun / la stânga / la dreapta.
- ⚠ De ce nu stochezi rangul direct: inserarea unui minim ar reactualiza tot arborele ($O(n)$).

Cheat-sheet consolidat

Concept	De reținut
Closest pair — brute force	$\Theta(n^2)$
Closest pair — D&C	$O(n \log n)$, $T(n)=2T(n/2)+O(n)$
Banda de verificare	lățime 2δ în jurul dreptei l
Câte puncte verifici	cel mult 8 în dreptunghi → 7 următoare
Cum eviți re-sortarea	presort $\Theta(n \log n)$ + împărțire (inversul lui MERGE)
Augmentare — 4 pași	structură → info → verifici menținerea → operații noi
Order statistics tree	RB tree + size ; $\text{size}[x] = \text{size}[L] + \text{size}[R] + 1$
OS-SELECT / OS-RANK	al i-lea / rangul, $O(\log n)$
Teorema augmentării	f din nod+copii în $O(1) \rightarrow O(\log n)$; se propagă la strămoși
De ce size, nu rang	rangul direct → inserare $O(n)$
Interval tree	RB tree + max ; cheia = $\text{low}[\text{int}[x]]$
$\text{max}[x]$	$\max(\text{high}[\text{int}[x]], \text{max}[L], \text{max}[R])$
INTERVAL-SEARCH	stânga dacă $\max[\text{left}] \geq \text{low}[i]$, altfel dreapta; $O(\log n)$

Concept	De reținut
Suprapunere intervale	$\text{low}[i] \leq \text{high}[i']$ și $\text{low}[i'] \leq \text{high}[i]$
Trichotomie	suprapun / i la stânga / i la dreapta

Bottom line: Partea 1 — closest pair e un divide-and-conquer $O(n \log n)$ cu un truc de combinare ingenios: după ce recursivitatea dă δ , orice pereche transfrontalieră mai bună stă în banda 2δ , iar acolo, sortat după y , verifici doar 7 puncte (fiindcă încap cel mult 8 în dreptunghiul $\delta \times 2\delta$); sortarea o ții prin presort + împărțire (inversul lui MERGE), nu re-sortezi. Partea 2 — augmentarea: adaugi un câmp la un RB tree și ai grijă să-l menții. Order-statistics tree adaugă size (subarbore) $\rightarrow$ OS-SELECT/OS-RANK în $O(\log n)$; interval tree adaugă max (capătul maxim) $\rightarrow$ INTERVAL-SEARCH care merge stânga dacă $\text{max}[\text{left}] \geq \text{low}[i]$. Teorema generală îți spune când e gratis: dacă f se calculează din nod + cei doi copii în $O(1)$, se menține în $O(\log n)$ fiindcă modificarea urcă doar pe lanțul de strămoși. Insight-ul de aur: stochezi size/max (info locală), NU rangul direct (info globală) $\rightarrow O(n)$ la inserare).